

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



COVID-19 HEALTH OUTCOME PREDICTION

Data Mining - IT160IU

Final Project Report

Members:

Pham Anh Khoi – ITCSIU23018
Pham Hoang Phuong – ITCSIU23056
Dang Ngoc Thai Son – ITCSIU23033

December 15, 2025

Contents

1	Introduction	3
2	Data Preprocessing Pipeline	4
2.1	Raw Data Overview	4
2.2	Preprocessing Steps for covid.csv	4
2.2.1	Implementation Stage	4
2.2.2	Results	6
3	Methodology - Classification/Prediction Algorithm	7
3.1	Objective	7
3.2	Model Selection	7
3.3	Implementation Process	7
3.3.1	Filtering Features	7
3.3.2	Handling Imbalance	7
3.3.3	Algorithms	8
3.3.4	Results	8
4	Improvement of Result	9
4.1	Analysis of Stage 1 Results: The Precision Bottleneck	9
4.2	Proposed Solution: Cascade Tunnel Architecture	9
4.2.1	Model Selection & Justification	9
4.2.2	Implementation Strategy	9
4.3	Results	10
5	Model Evaluation	11
5.1	Stage 1: Naive Bayes Summary	11
5.2	Final Pipeline: Cascade Tunnel Summary	11
5.3	Comparative Analysis	12

Abstract

This report presents the development of a machine learning pipeline which is designed to predict mortality risk among patients diagnosed with COVID-19, with the goal of spotting risks early and make sure healthcare resources go where they're needed most. Using a dataset from the Mexican government that contains more than one million patient records, we built a predictive model based on demographic information, key comorbidities, and clinical indicators such as pneumonia and ICU admission. In detail, we analyzed variables such as patient age, pneumonia status, and ICU-related features, which are among the most influential predictors of mortality. This work illustrates the potential of applying machine learning to large-scale patient data to create a reliable mortality prediction tool, offering practical value for managing clinical decision-making during public health crises.

1 Introduction

The COVID-19 pandemic has triggered a global shock, placing extreme pressure on healthcare systems worldwide. A key challenge of the SARS-CoV-2 virus lies in the wide variation in clinical progression: while most individuals experience mild symptoms, a considerable proportion develops severe illness and faces a high risk of mortality. This disparity created an urgent problem for frontline providers, that is how to rapidly and accurately classify and triage patients to prioritize limited treatment resources.

In overloaded hospital environments, early identification of patients at high risk for mortality is crucial. Early risk stratification not only helps optimize the use of scarce resources, such as Intensive Care Unit (ICU) beds and ventilators, but also enables medical teams to implement timely treatment protocols, significantly improving survival chances. Traditional assessment methods, though valuable, often lack the speed and consistency required during a large-scale pandemic.

In the era of data-driven medicine, Machine Learning (ML) has emerged as a powerful tool to address this challenge. By analyzing large datasets containing patient health information, ML models can uncover complex patterns and correlations that are difficult for humans to detect, providing objective, evidence-based predictions to support critical clinical decision-making.

The primary objective of this project is to build and evaluate a complete ML pipeline for predicting mortality risk among COVID-19 patients. To achieve this, we utilize a large public dataset provided by the Mexican government, which includes anonymized information from over one million patients. The model is trained on input features such as demographic data (age, sex), pre-existing medical conditions (diabetes, hypertension, obesity, etc.), and initial clinical status.

This report is structured as follows: the next section provides a detailed description of the dataset and the preprocessing stage. Section three explains the methodology, including preprocessing techniques and model architecture. Section four analyzes the experimental results and model performance. Finally, section five presents the conclusions, discusses the implications of the findings, and proposes future directions for development.

2 Data Preprocessing Pipeline

2.1 Raw Data Overview

The raw data for the pipeline is sourced from the large and complex file (`covid.csv`), which is located in the `/data` directory.

Initial Characteristics of `covid.csv`

This dataset is characterized by significant inconsistencies and the presence of missing value codes that must be addressed before moving to the next stage.

Table 1: Data Description Before Pre-processing

Attribute	Values
USMER	[2, 1]
MEDICAL_UNIT	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
SEX	[1, 2]
PATIENT_TYPE	[1, 2]
DATE_DIED	['03/05/2020', '03/06/2020', '9999-99-99', ...] (401 unique values)
INTUBED	[97, 1, 2, 99]
PNEUMONIA	[1, 2, 99]
AGE	[65, 72, 55, ...] (121 unique values)
PREGNANT	[2, 97, 98, 1]
DIABETES	[2, 1, 98]
COPD	[2, 1, 98]
ASTHMA	[2, 1, 98]
INMSUPR	[2, 1, 98]
HIPERTENSION	[1, 2, 98]
OTHER_DISEASE	[2, 1, 98]
CARDIOVASCULAR	[2, 1, 98]
OBESITY	[2, 1, 98]
RENAL_CHRONIC	[2, 1, 98]
TOBACCO	[2, 1, 98]
CLASIFFICATION_FINAL	[3, 5, 7, 6, 1, 2, 4]
ICU	[97, 2, 1, 99]

- *Missing Values Codes:* Placeholder codes 97, 98, and 99 are used across multiple columns (e.g., INTUBED, PREGNANT, ICU) to denote missing or unknown information.
- *Binary Encoding:* Binary features use an inconsistent encoding of 1 (YES) and 2 (NO), which must be standardized to 1 and 0.
- *Target Variable Encoding:* The patient outcome information is implicitly encoded in the DATE_DIED column. The value 9999-99-99 signifies the patient is alive, while any specific date indicates death.
- *Non-COVID Records:* The dataset includes records where CLASIFFICATION_FINAL indicates the patient was not confirmed to have COVID-19, which also need to be filtered out.

2.2 Preprocessing Steps for `covid.csv`

2.2.1 Implementation Stage

The following sequence of steps ensures the data is clean, standardized, and ready for model training:

1. *Filter Non-COVID Patients:* Remove records not confirmed to have COVID-19 using the `CLASIFFICATION_FINAL` attribute.
2. *Feature Extraction (DIED):* Create a new binary target feature, `DIED`, from the `DATE_DIED` column. A value of 1 indicates death (date present), and 0 indicates alive (9999-99-99).
3. *Remove Redundant Columns:* Drop the original `DATE_DIED` and `CLASIFFICATION_FINAL` attributes.
4. *Standardize Binary Values:* Convert the dataset's binary encoding from 1 (YES) and 2 (NO) to the standard 1 (YES) and 0 (NO).
5. *Mark Missing Values:* Convert the placeholder codes (97, 98, 99) to standard missing value markers.
6. *Impute Missing Values:* A sophisticated group-based mode imputation is performed:
 - For each column with missing values, a **Chi-Square Test** is used to find the most correlated attribute.

$$X^2 = \sum \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}}$$
 - Missing values are imputed using the mode (most frequent value) calculated within groups defined by the most correlated column.
 - Any remaining missing values are filled using the global mode of the respective column.
7. *Create Age Groups:* A new nominal feature, `AGE_GROUP`, is created by binning the `AGE` attribute into six categories:
 - 0-17 (Pediatric)
 - 18-39 (Young Adult - Low Risk)
 - 40-49 (Adult - Moderate Risk)
 - 50-59 (Late Adult - High Risk)
 - 60-69 (Senior - Very High Risk)
 - 70+ (Elderly - Critical Risk)
8. *Finalize and Export:* The fully cleaned data is prepared as an `Instances` object and saved to `.csv` and `.arff` formats in the `/data/preprocess` directory.

2.2.2 Results

Table 2: Data Description After Pre-processing

Attribute	Values and Description
USMER	[0, 1]; 0 = First Level Unit, 1 = Second Level Unit.
MEDICAL_UNIT	[1, 0, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13] (13 unique values).
SEX	[1, 0]; 1 = Female, 0 = Male.
PATIENT_TYPE	[1, 0]; 1 = Home, 0 = Hospitalized.
INTUBED	[0, 1]; Binary (0=No, 1=Yes).
PNEUMONIA	[1, 0]; Binary (1=Yes, 0=No).
AGE	[0, 1, 3 ... 121] (117 unique values). Numeric.
PREGNANT	[0, 1]; Binary (0=No, 1=Yes).
DIABETES	[0, 1]; Binary (0=No, 1=Yes).
COPD	[0, 1]; Binary (0=No, 1=Yes).
ASTHMA	[0, 1]; Binary (0=No, 1=Yes).
INMSUPR	[0, 1]; Binary (0=No, 1=Yes).
HIPERTENSION	[0, 1]; Binary (0=No, 1=Yes).
OTHER_DISEASE	[0, 1]; Binary (0=No, 1=Yes).
CARDIOVASCULAR	[0, 1]; Binary (0=No, 1=Yes).
OBESITY	[0, 1]; Binary (0=No, 1=Yes).
RENAL_CHRONIC	[0, 1]; Binary (0=No, 1=Yes).
TOBACCO	[0, 1]; Binary (0=No, 1=Yes).
ICU	[0, 1]; Binary (0=No, 1=Yes).
DIED	[0, 1]; Target Variable (0=Survived, 1=Died).
AGE_GROUP	['0-17', '18-39', '40-49', '50-59', '60-69', '70+'] (6 unique values); Nominal.

3 Methodology - Classification/Prediction Algorithm

3.1 Objective

The primary objective of this phase is to implement a robust classification model to accurately predict the mortality risk of COVID-19 patients (Target: DIED). By computing the efficiency and probabilistic output of the Naive Bayes algorithm, the goal is to establish a high-performance predictive tool that serves as the best available solution for identifying critical clinical outcomes.

3.2 Model Selection

For this purpose, the Naive Bayes classifier was selected to be our prediction model. Naive Bayes is a probabilistic classifier based on Bayes' theorem with an assumption of independence between features. It was chosen since the algorithm is particularly suitable for medical datasets with high-dimensional categorical attributes and is computationally efficient for large-scale data like the current dataset.

3.3 Implementation Process

3.3.1 Filtering Features

This module is responsible for constructing the specific dataset used for training and testing the Naive Bayes Classifier. Utilizing preprocessed data from the `/data/preprocess` directory, the goal is to create a consolidated dataset suitable for the model's architectural role. From the available attributes, we selected a focused subset of seven features: "SEVERITY_INDEX", "AGE_GROUP", "SEX", "PNEUMONIA", "COPD", "ASTHMA", "TOBACCO", and the target variable DIED. The reasons for selecting these attributes are:

1. *The Independence Assumption:* Naive Bayes assumes that every feature is mathematically independent of the others. In the raw data, attributes like INTUBED, ICU, and PATIENT_TYPE are highly correlated (e.g., an intubated patient is almost certainly in the ICU). Including all of them would cause the model to "double count" or "triple count" the risk signal, violating the algorithm's probability rules. We replaced these correlated columns with the single SEVERITY_INDEX to provide a clean signal (0-3) without breaking the independence assumption.
2. *The Sparsity Problem:* Naive Bayes relies on probability tables for feature combinations. With too many features (e.g., 22), the table becomes massive, and specific combinations of rare attributes might have zero examples in the training set (Zero Frequency error). Keeping the feature set small ensures that every combination is robustly represented in the data.

3.3.2 Handling Imbalance

The dataset is significantly imbalanced, with survivors (DIED=0) accounting for 93% of records versus only 7% for deaths (DIED=1). To address this, a strategic **Undersampling** workflow is applied:

- *Data Splitting:* First, 20% of the data is separated as a distinct Test Set to evaluate performance on real-world distributions.
- *Balancing Training Data:* From the remaining 80% (Training Set), we retain all instances of the minority class ("Died") and randomly select an equal number of instances from the majority class ("Cured").
- *Result:* This creates a training dataset with a 50/50 ratio, forcing the Naive Bayes model to treat mortality cases as equally important as survival cases during the learning process.

3.3.3 Algorithms

The classification model is implemented using the Naive Bayes algorithm provided by the Weka library `weka.classifiers.bayes.NaiveBayes`).

A distinct feature of this implementation is the application of strategic undersampling inside the validation loop. The process addresses class imbalance without causing data leakage:

1. *Fold Splitting:* In each iteration, the data is split into a Training Set and a Test Set (10-Fold Cross-Validation).
2. *Training Data Balancing:* The `strategicUndersample` function is applied exclusively to the Training Set. This balances the distribution to 50/50 (Died vs. Survived), forcing the Naive Bayes model to learn the characteristics of the minority class effectively.
3. *Realistic Evaluation:* The model is evaluated on the unmodified Test Set, which retains the original "real-world" imbalance (93% Survived vs. 7% Died). This ensures that the reported metrics reflect the model's true performance in a clinical setting.

3.3.4 Results

1	=== Summary ===		
2	Correctly Classified Instances	932058	88.8881 %
3	Incorrectly Classified Instances	116517	11.1119 %
4	Kappa statistic	0.3889	
5	Mean absolute error	0.1289	
6	Root mean squared error	0.3042	
7	Relative absolute error	25.7797 %	
8	Root relative squared error	60.8401 %	
9	Total Number of Instances	1048575	

Explanation:

- *Overall Accuracy:* The model achieved an accuracy of 88.89%, correctly classifying over 930,000 instances. This indicates a strong capability to handle the large-scale dataset effectively.
- *Kappa Statistic:* A value of 0.39 suggests moderate agreement. This reflects the challenge of predicting on a highly imbalanced test set but confirms the model performs significantly better than random chance.
- *Error Rates:* The Mean Absolute Error (MAE) is low at 0.1289, implying that the predicted probability scores are generally close to the actual class labels.

4 Improvement of Result

4.1 Analysis of Stage 1 Results: The Precision Bottleneck

Upon analyzing the performance metrics from the Naive Bayes experiment above, a critical operational flaw emerged despite the high overall accuracy:

- *High Recall vs. Low Precision:* The experimental results show a distinct trade-off. While the Naive Bayes model successfully identified the majority of deceased patients (High Recall), the Precision score remains suboptimal. In data terms, this means that to catch the true "Death" cases, the model had to flag a very large number of "Survival" cases as "At Risk." It effectively casts a "wide net" but lacks the sharpness to filter the catch.
- *The Consequence of False Positives:* The Confusion Matrix reveals a high number of False Positives (Survivors incorrectly predicted as Deceased). In a real-world hospital scenario, this creates a "False Alarm" crisis. If we rely solely on this model, medical staff would be forced to allocate scarce resources (ICU beds, ventilators) to patients who don't actually need them. The single model proves to be a "blunt instrument"—excellent for initial screening, but too imprecise for final triage decisions.

4.2 Proposed Solution: Cascade Tunnel Architecture

To fix this specific imbalance—keeping the High Recall from Stage 1 while eliminating the False Positives—we implemented a Cascade Tunnel Architecture. Instead of forcing one model to do everything, we split the job into a two-step pipeline:

4.2.1 Model Selection & Justification

1. Stage 1: Naive Bayes (The Screener)

- *Role:* The "General Practitioner" or Gatekeeper.
- *Why we kept it:* Despite the precision issues mentioned above, Naive Bayes is incredibly fast and has excellent sensitivity. We use it exactly for its strength: to rapidly filter out the massive number of obviously safe patients. If it has any doubt, it flags the patient, ensuring no critical case is left behind.

2. Stage 2: Random Forest (The Specialist)

- *Role:* The Critical Care Specialist.
- *Why we added it:* To solve the False Positive problem found in Stage 1. Random Forest is much better at understanding complex, non-linear relationships (e.g., how age combines with specific comorbidities). Crucially, we only show it the "Suspects" flagged by Stage 1. This allows it to focus 100% of its power on distinguishing between the "False Alarms" and the "True Critical" cases, effectively cleaning up the results.

4.2.2 Implementation Strategy

The logic comes directly from the `CascadeTunnelCrossValidation` module and follows these four steps:

1. *Step 1: The Broad Screen (Stage 1):* We train the Naive Bayes model on a balanced dataset (50% Survived / 50% Died) so it pays attention to rare death cases. It screens every single patient entering the system.
2. *Step 2: The Handoff:* We filter the patients based on the Stage 1 result:
 - If Stage 1 says 0 (Safe): The patient is low-risk. We stop here (Efficiency gain).
 - If Stage 1 says 1 (Risk): The patient is marked as a "Suspect" and sent to the Specialist (Stage 2).

3. *Step 3: Specialist Review (Stage 2):* The Random Forest looks at the suspects.
 - Setup: We limit the forest to 20 trees and a depth of 10 to keep it focused.
 - Prioritizing Critical Cases: Instead of undersampling (which might lose data), we use weighting. We tell the model that missing a death (Weight 2.0) is twice as bad as a false alarm (Weight 1.0). This forces the Specialist to be very careful before dismissing a risk.
4. *Step 4: Merging the Results:* The final prediction combines both stages. For the "Suspects," the Random Forest has the final say. It effectively "overrules" the Naive Bayes, removing the false alarms while keeping the true critical cases.

4.3 Results

1	=== Summary ===		
2	Correctly Classified Instances	981233	93.5778 %
3	Incorrectly Classified Instances	67342	6.4222 %
4	Kappa statistic	0.4358	
5	Mean absolute error	0.0745	
6	Root mean squared error	0.1758	
7	Relative absolute error	14.8996 %	
8	Root relative squared error	35.1630 %	
9	Total Number of Instances	1048575	

Explanation:

- *Overall Accuracy:* The Cascade Tunnel achieved a superior accuracy of 93.58%, correctly classifying over 981,000 instances. This represents a net improvement of approximately 4.7% over the single-stage model, significantly reducing the number of misclassified cases from 116,517 to 67,342.
- *Kappa Statistic:* The Kappa value increased to 0.4358. This improvement signifies a higher level of agreement between the predicted and observed classes, confirming that the addition of the Random Forest layer effectively refined the classification logic beyond the baseline probability.
- *Error Rates:* Both error metrics showed substantial improvement. The Mean Absolute Error (MAE) dropped to 0.0745, and the Root Mean Squared Error (RMSE) decreased to 0.1758. This reduction indicates that the Cascade model's probability estimates are much sharper and closer to the true binary labels compared to the fuzzier probabilistic outputs of the Naive Bayes stage.

5 Model Evaluation

This section evaluates the performance of the standalone Naive Bayes model (Stage 1) versus the final Cascade Tunnel architecture using 10-Fold Cross-Validation.

5.1 Stage 1: Naive Bayes Summary

Figure 1 illustrates the performance of the initial screening layer.

```
--- FINAL 10-FOLD SUMMARY (STAGE 1) ---  
Mean Accuracy: 88.89% (+/- 0.08%)  
Mean Recall: 92.29% (Target: >90%)  
Mean Precision: 39.10% (Expected: Low)  
Mean F1-Score: 0.5493
```

Figure 1: Naive Bayes Summary (Stage 1)

The results confirm the model's role as a "Screener":

- *High Recall (92.29%)*: The model successfully identified over 92% of actual mortality cases, meeting our safety target (> 90%). It rarely misses a critical patient.
- *Low Precision (39.10%)*: To achieve high recall, the model generates many False Positives. Only 39% of patients flagged as "High Risk" actually died, indicating a high "false alarm" rate that would burden hospital resources.

5.2 Final Pipeline: Cascade Tunnel Summary

Figure 2 shows the results after passing "suspects" through the Random Forest specialist.

```
--- FINAL 10-FOLD SUMMARY (CASCADE TUNNEL) ---  
Mean Accuracy: 93.58% (+/- 0.07%)  
Mean Recall: 69.04% (Target: >90%)  
Mean Precision: 54.97%  
Mean F1-Score: 0.6120  
  
--- Cascade Tunnel Performance Summary ---  
Average Stage 1 Build Time: 0.0230 seconds  
Average Stage 2 Build Time: 0.4878 seconds  
Total Average Build Time: 0.5108 seconds
```

Figure 2: Cascade Tunnel Summary (Final Output)

The Cascade architecture significantly altered the performance profile:

- *Improved Precision (+15.8%)*: Precision rose to 54.97%. The Random Forest successfully filtered out the "false alarms," distinguishing true critical cases from survivors with similar symptoms.
- *Higher Accuracy (+4.7%)*: Overall accuracy increased to **93.58%**, reflecting a more reliable classification of the majority class.
- *Recall Trade-off*: Recall dropped to 69.04%. The stricter filtering made the model more conservative, re-classifying ambiguous cases as survivors.

5.3 Comparative Analysis

The transition from Stage 1 to the Cascade Tunnel reveals a distinct trade-off between sensitivity and efficiency:

1. *Noise Reduction*: The Cascade Tunnel successfully solved the precision bottleneck of Stage 1. By allowing the Random Forest to re-evaluate risk, we reduced the number of patients incorrectly flagged as critical, which is vital for efficient resource allocation (ICU beds/ventilators).
2. *The Cost of Specificity*: The drop in Recall (92% $\rightarrow$ 69%) is the direct cost of eliminating false positives. The Random Forest detected that many elderly/pneumonia patients did survive and classified similar future cases as "Safe." While this increases the F1-Score (0.6120), it means the final model is less of a "safety net" than the raw Naive Bayes, but far more accurate for triage decisions.